Agent Reliability Lab

Controlled Fault Injection and Failure Analysis for AI Agents

Department of Computer Science and Engineering

(Supervisor: Mr. Pradeep Kumar)



Design/Implementation Phase Progress Review | Group CSE/CSPROJECT/2027/G053 | 17 October 2026

Name of Member	Roll Number	Role
Suryansh Singh	2301330100208	Team Lead: architecture, harness, fault injection, store, CLI
Simran Kumari	2301330100200	Metrics, failure analysis, statistics, optimization
Tanishka Sahu	2301330100210	Task suites, baseline agents, reports, testing

Index



- Problem, Idea and Objectives
- Proposed System
- Software Requirement Analysis and Modules
- Design: Class, Sequence, DFD, ER, Gantt
- Implementation: Algorithms and Code Snapshots
- Implementation Output and Preliminary Results
- Research Paper and Work Status
- References

Problem, Idea and Objectives

The problem

- AI agents now plan, call tools, and retry, yet benchmarks report one number: did the agent finish?
- That number hides why runs fail, how consistent the agent is, and what reliability costs.

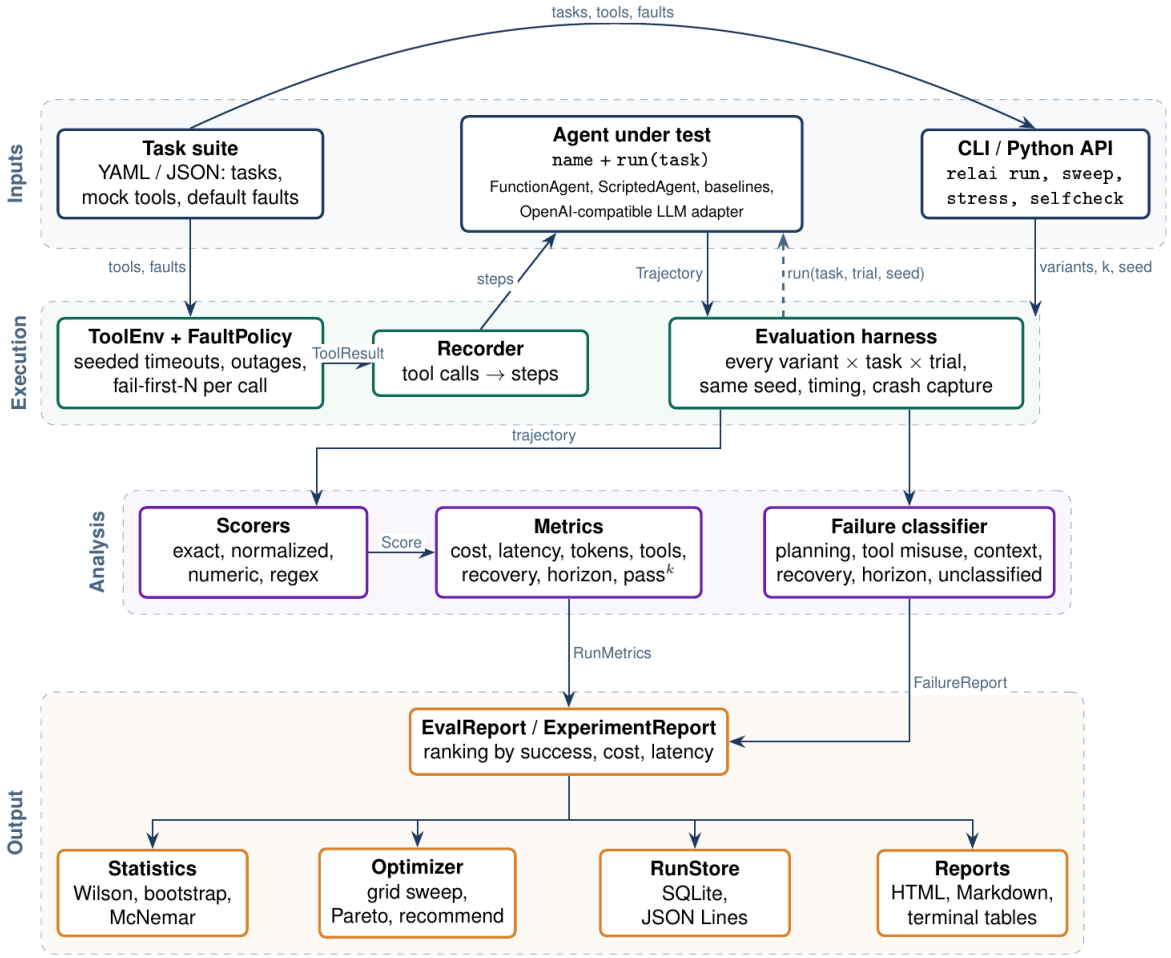
Our idea: a crash-test lab for AI agents

- Break tools on purpose, the same way for every agent, and run every task several times.
- Explain each failed run with evidence, then pick the cheapest setup that stays reliable.

Objectives and status

- O1 Evaluate: success, pass^k, consistency, cost, latency, recovery, steps (done)
- O2 Explain: five failure modes with evidence, seeded fault injection (done)
- O3 Optimize: paired tests, Pareto front, cheapest reliable setting (done; live LLM study next)

Proposed System



Four layers

- Inputs: YAML task suites with mock tools and faults, and any agent with name + run(task)
- Execution: the harness runs every variant x task x trial; ToolEnv injects seeded faults
- Analysis: scorers, metrics, and the failure classifier only read trajectories
- Output: ranking with confidence intervals, paired test, Pareto sweep, SQLite store, HTML report

Runs anywhere

- Offline, CPU only, no API key; live LLMs plug in through an OpenAI-compatible adapter

Software Requirement Analysis

Functional requirements

- FR1 Load suites (YAML/JSON) with tools, faults, step budgets
- FR2 Run any agent through name + run(task)
- FR3 Inject reproducible timeouts, outages, fail-first faults
- FR4 Repeat tasks k times; report pass@k, pass^k, consistency
- FR5 Score answers: exact, normalized, numeric, regex
- FR6 Diagnose failures with evidence; keep unknowns unclassified
- FR7 Compare variants with intervals and a paired test
- FR8 Sweep settings; return Pareto front and recommendation
- FR9 Store runs (SQLite, JSONL); render HTML and Markdown
- FR10 Offer a CLI and a Python API

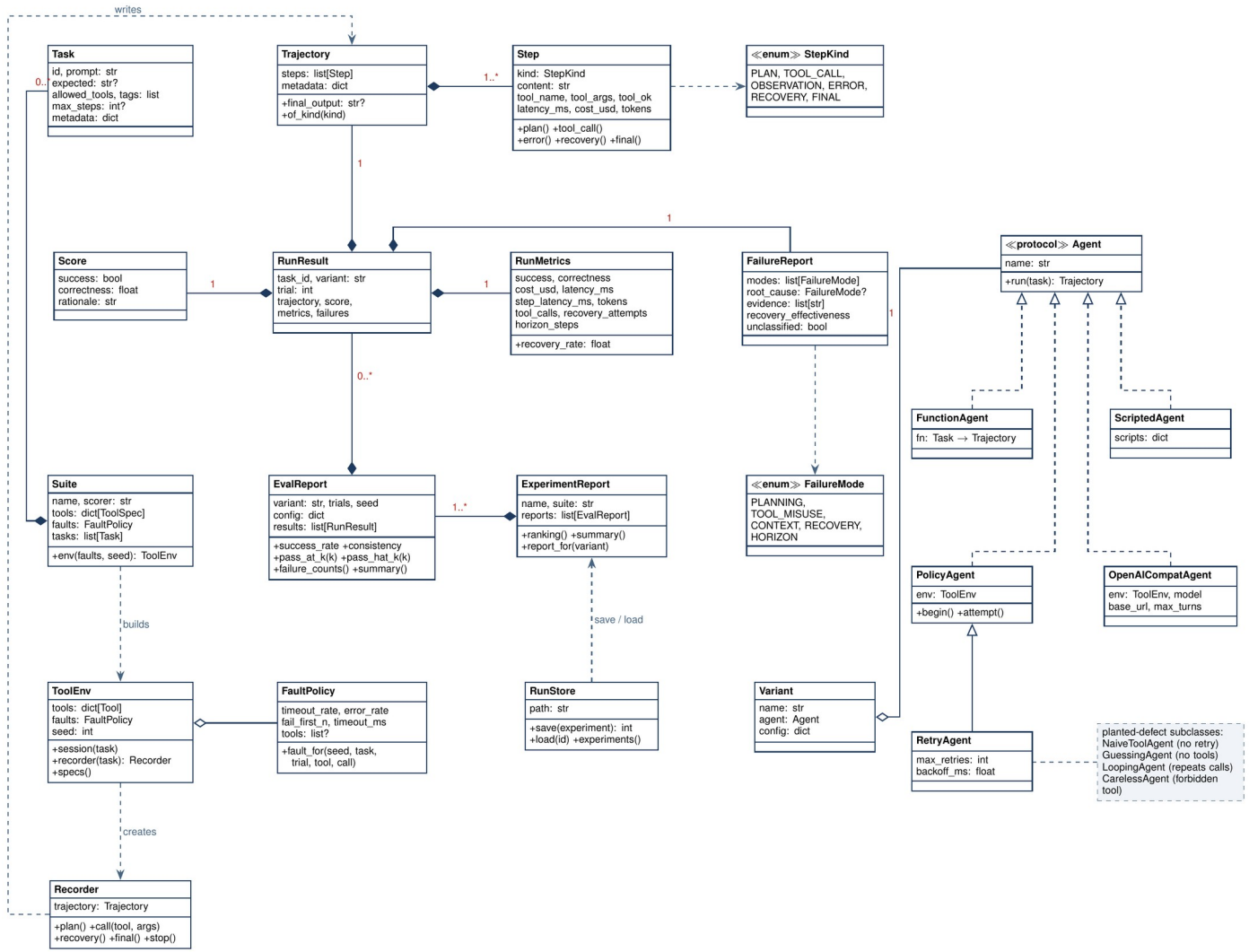
Non-functional requirements

- Reproducibility: same seed, same faults, same numbers
- Fairness: every variant meets the identical fault schedule
- Performance: about 6,500 scored runs per second; 81 tests in under 1 s
- Portability: Python 3.11 to 3.13, any OS, 2 runtime dependencies
- Extensibility: new agents, scorers, suites, modes without harness changes
- Robustness: an agent crash becomes a failed run, not a crashed experiment
- Security: no eval(), escaped HTML, offline by default
- Maintainability: typed models, lint, CI on three Python versions

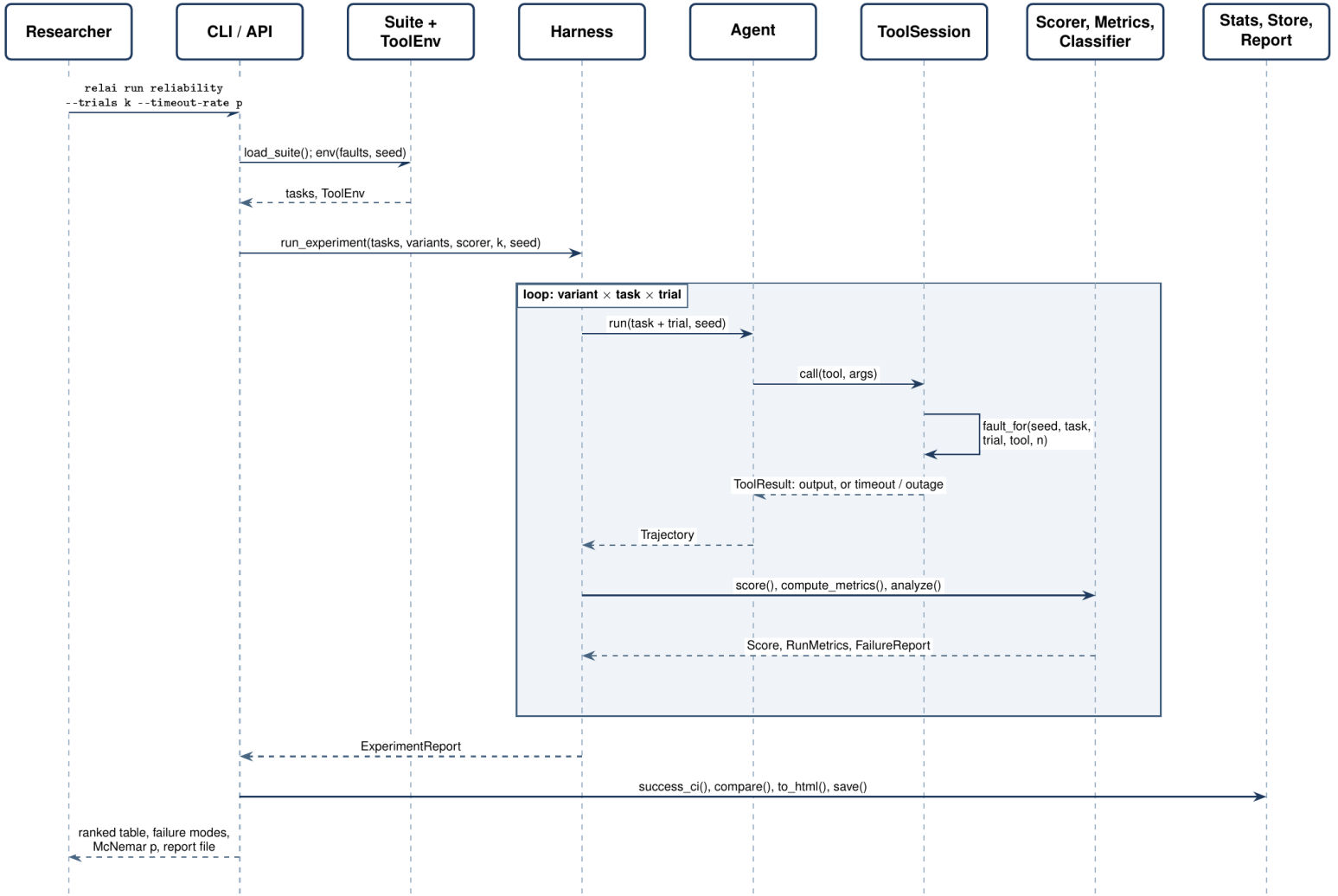
Major Modules and Functionalities

Module	What it does	Owner
Domain types and agent seam	Task, Step, Trajectory; FunctionAgent, ScriptedAgent	Suryansh
Tool environment	Tools, seeded FaultPolicy, Recorder of steps	Suryansh
Evaluation harness	Trials, timing, crash capture, pass@k, pass^k, experiments	Suryansh
Metrics and scorers	Cost, latency, tokens, recovery, horizon; five scorers	Simran
Failure analysis	Five modes, root cause, evidence, unclassified	Simran
Statistics and optimizer	Wilson, bootstrap, McNemar; sweep, Pareto, recommend	Simran
Suites and baseline agents	YAML suites, planted-defect agents, classifier self-check	Tanishka
Reports and tests	HTML and Markdown reports, 81 tests, examples	Tanishka
Store, CLI, LLM adapter	SQLite and JSONL, relai commands, OpenAI-compatible agent	Suryansh

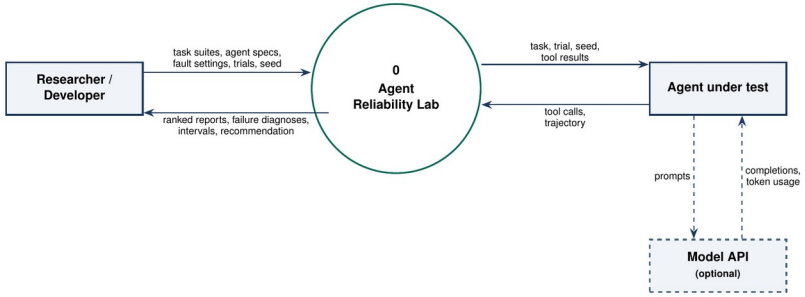
Class Design



Sequence Diagram: relai run

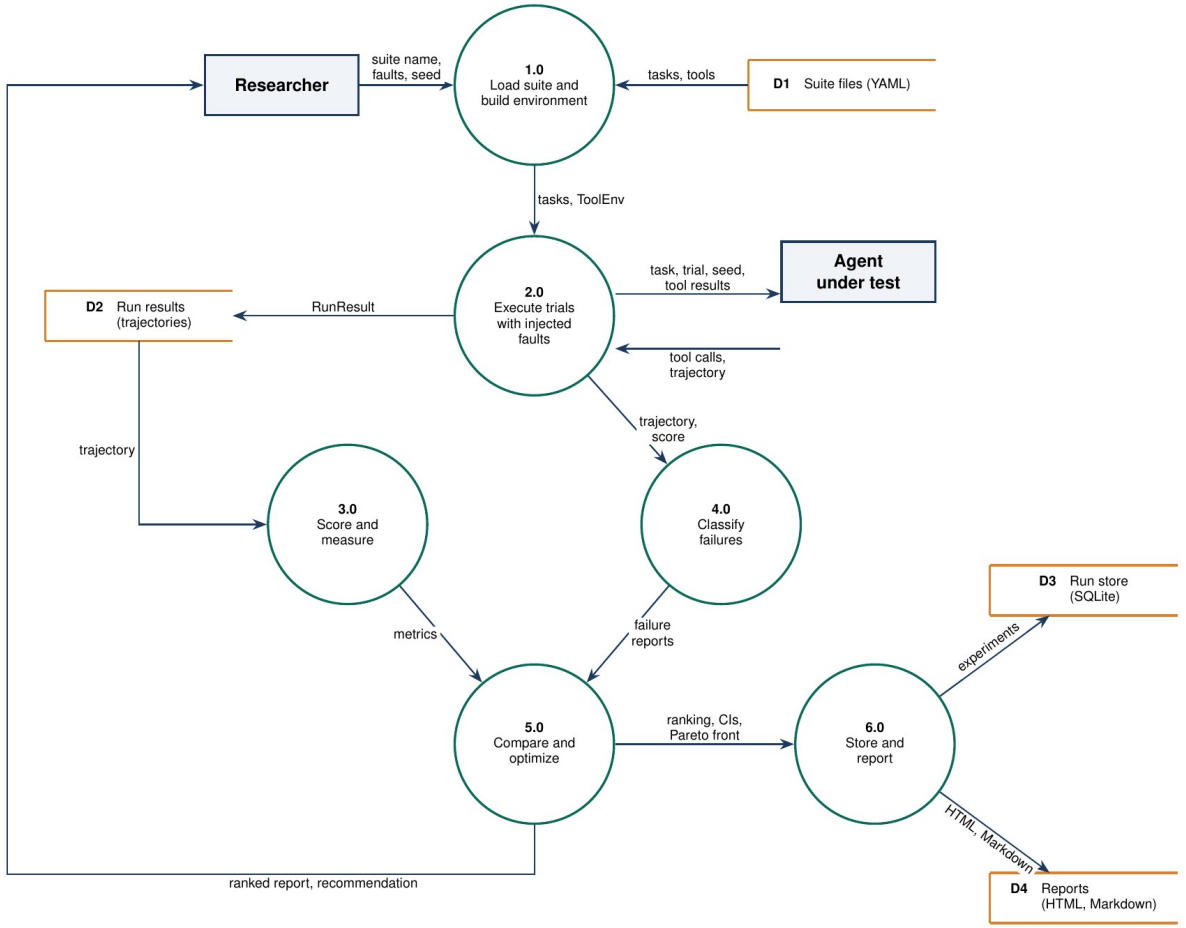


Data Flow Diagrams: Level 0 and Level 1

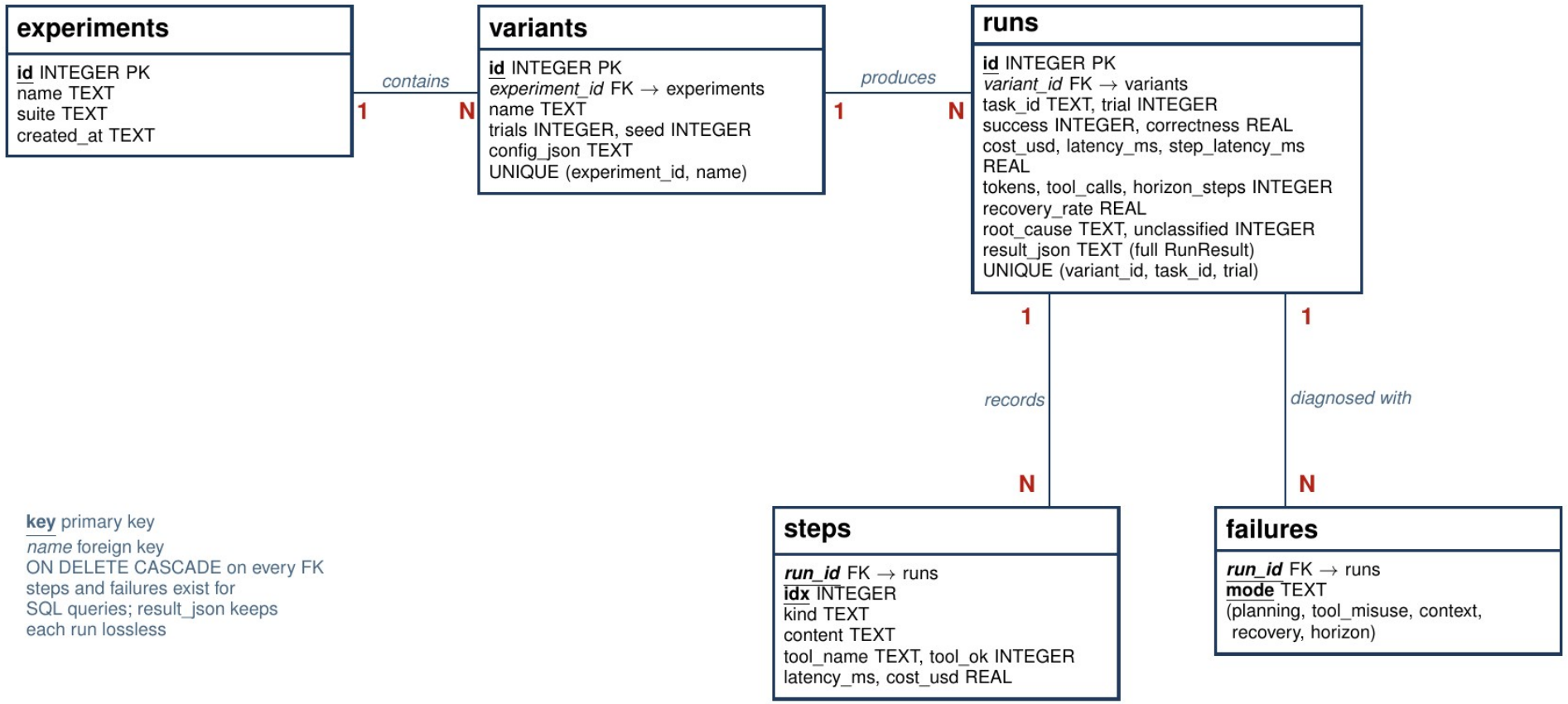


Level 0: context diagram

Level 1: six processes and four data stores. Analysis reads stored run results, never the agent.

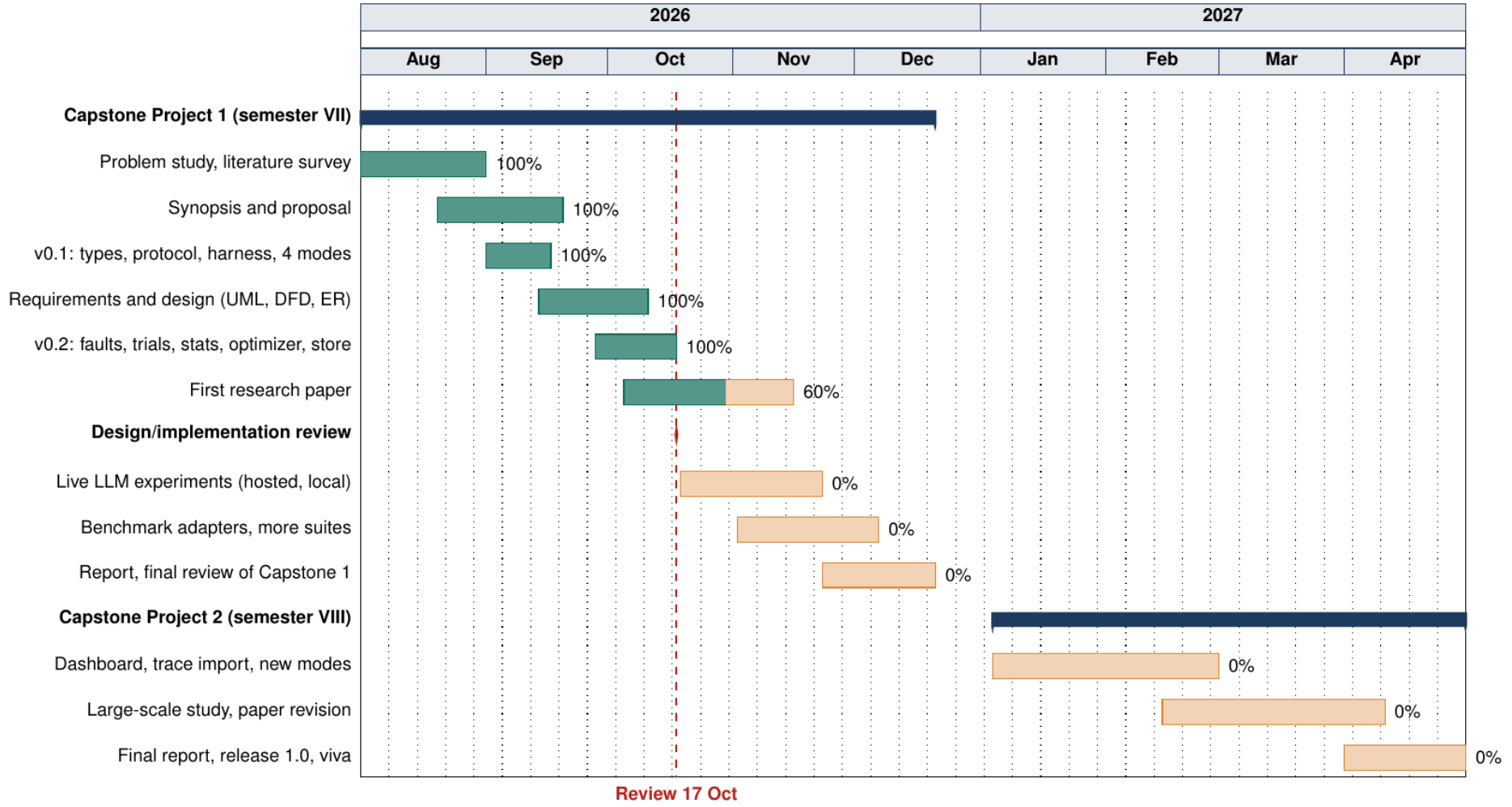


Database Design: ER Diagram



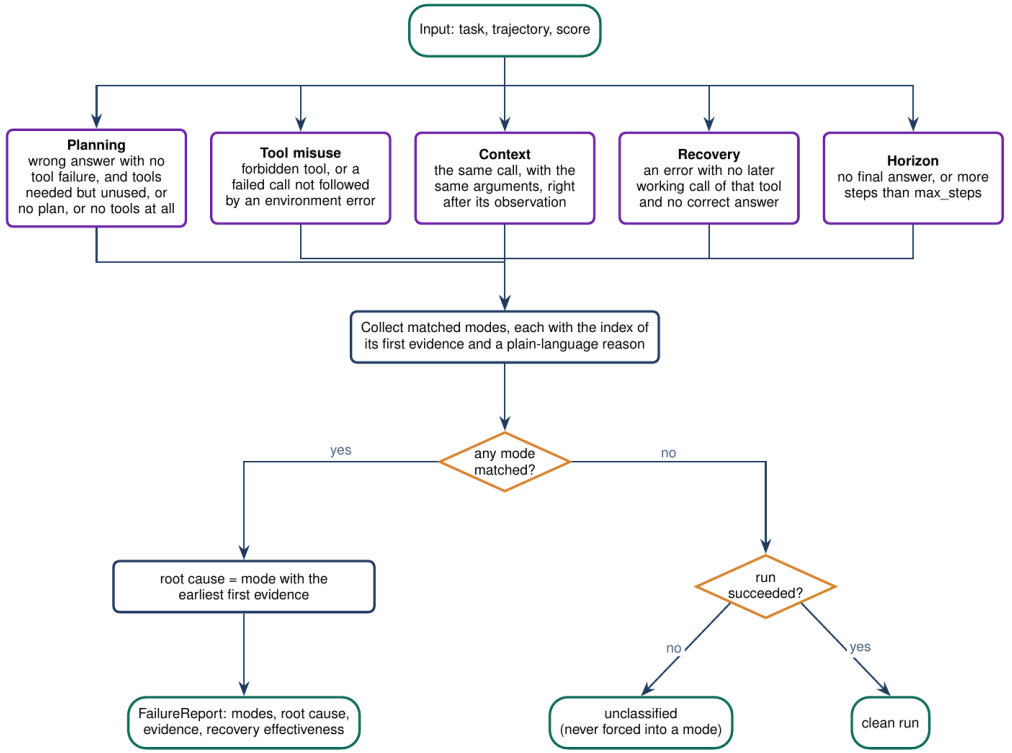
SQLite run store: experiments contain variants, variants produce runs, runs record steps and failure modes. result_json keeps each run lossless.

Gantt Chart


2026
2027
Aug
Sep
Oct
Nov
Dec
Jan
Feb
Mar
Apr
Capstone Project 1 (semester VII)
Problem study, literature survey
100%
Synopsis and proposal
100%
v0.1: types, protocol, harness, 4 modes
100%
Requirements and design (UML, DFD, ER)
100%
v0.2: faults, trials, stats, optimizer, store
100%
First research paper
60%
Design/implementation review
Live LLM experiments (hosted, local)
0%
Benchmark adapters, more suites
0%
Report, final review of Capstone 1
0%
Capstone Project 2 (semester VIII)
Dashboard, trace import, new modes
0%
Large-scale study, paper revision
0%
Final report, release 1.0, viva
0%
Review 17 Oct

Implementation of Core Algorithms

- Fault schedule: u = hash(seed, task, trial, tool, call n); timeout if u < p. Every agent meets the same faults.
- Repeated trials: pass^k = C(c,k) / C(n,k) for c passes in n trials; pass@k = 1 - C(n-c,k) / C(n,k).
- Classification: five detectors; root cause = mode with the earliest evidence; no match means unclassified.
- Paired test: exact McNemar on runs that only one variant passed.
- Optimization: keep the Pareto front over success, cost, latency; recommend the cheapest setting that meets the target.



Code Snapshots: Fault Injection and Retry

src/relai/env.py (line 46)

```python
def fault_for(self, *, seed: int, task_id: str, trial: int, tool: str, call: int) -> str | None:
    """Return the fault for the call-th call of a tool in one run, if any."""
    if not self.applies_to(tool):
        return None
    if call <= self.fail_first_n:
        return "timeout"
    draw = random.Random(f"{seed}:{task_id}:{trial}:{tool}:{call}").random()
    if draw < self.timeout_rate:
        return "timeout"
    if draw < self.timeout_rate + self.error_rate:
        return "unavailable"
    return None
```

src/relai/baselines.py (line 91)

```python
def attempt(self, rec: Recorder, tool: str, args: dict[str, Any]) -> ToolResult:
    result = rec.call(tool, args)
    retries = 0
    while not result.ok and result.fault is not None and retries < self.max_retries:
        retries += 1
        rec.recovery(
            f"retry {retries}/{self.max_retries} after {result.fault}",
            latency_ms=self.backoff_ms * 2 ** (retries - 1),
        )
        result = rec.call(tool, args)
    return result
```

Code Snapshots: Evaluation Loop and Statistics

src/relai/harness.py (line 141)

```python
def evaluate(
    agent: Agent,
    tasks: Sequence[Task],
    scorer: Scorer = exact_match,
    *,
    trials: int = 1,
    seed: int = 0,
) -> EvalReport:
    """Run one agent on every task, trials times each, and return scored reports.

    The trial index and seed reach the agent through task.metadata, so tool
    environments can replay the same fault schedule for every variant.
    """
    if trials < 1:
        raise ValueError("trials must be at least 1")
    results: list[RunResult] = []
    for task in tasks:
        for trial in range(trials):
            run_task = task.model_copy(
                update={"metadata": {**task.metadata, TRIAL_KEY: trial, SEED_KEY: seed}},
                deep=True,
            )
            started = time.perf_counter()
            trajectory = _run_safely(agent, run_task)
            wall_ms = (time.perf_counter() - started) * 1000
            score = scorer(task, trajectory)
            metrics = compute_metrics(trajectory, score, wall_ms)
            failures = analyze(task, trajectory, score)
            results.append(
                RunResult(
                    task_id=task.id,
                    variant=agent.name,
                    trial=trial,
                    trajectory=trajectory,
                    score=score,
                    metrics=metrics,
                    failures=failures,
                )
            )
    return EvalReport(variant=agent.name, trials=trials, seed=seed, results=results)
```

src/relai/harness.py (line 198)

```python
def _pass_at_k(n: int, c: int, k: int) -> float:
    _check_k(n, k)
    if n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k) / comb(n, k)
```

src/relai/harness.py (line 205)

```python
def _pass_hat_k(n: int, c: int, k: int) -> float:
    _check_k(n, k)
    return comb(c, k) / comb(n, k)
```

src/relai/stats.py (line 80)

```python
def mcnemar_exact(a_only: int, b_only: int) -> float:
    """Two-sided exact McNemar p-value from the two kinds of discordant pairs."""
    n = a_only + b_only
    if n == 0:
        return 1.0
    tail = sum(comb(n, i) for i in range(min(a_only, b_only) + 1)) / 2**n
    return min(1.0, 2 * tail)
```

Implementation Output

Terminal

```
$ relai run reliability --agent naive --agent retry:1 --agent retry:3 --trials 5 --timeout-rate 0.3
variant  success  95% CI        pass^k  consistent  cost     latency  recovery  horizon
-------  -------  ------------  ------  ----------  -------  -------  --------  -------
retry:3  1.00     [1.00, 1.00]  1.00    1.00        $0.0030  1116ms   1.00      8.8
retry:1  0.77     [0.67, 0.88]  0.33    0.33        $0.0025  947ms    0.79      7.2
naive    0.58     [0.38, 0.77]  0.25    0.33        $0.0016  543ms    0.58      5.0

failure modes (retry:3): {'horizon': 12}
failure modes (retry:1): {'recovery': 14, 'horizon': 2}
failure modes (naive): {'recovery': 25}

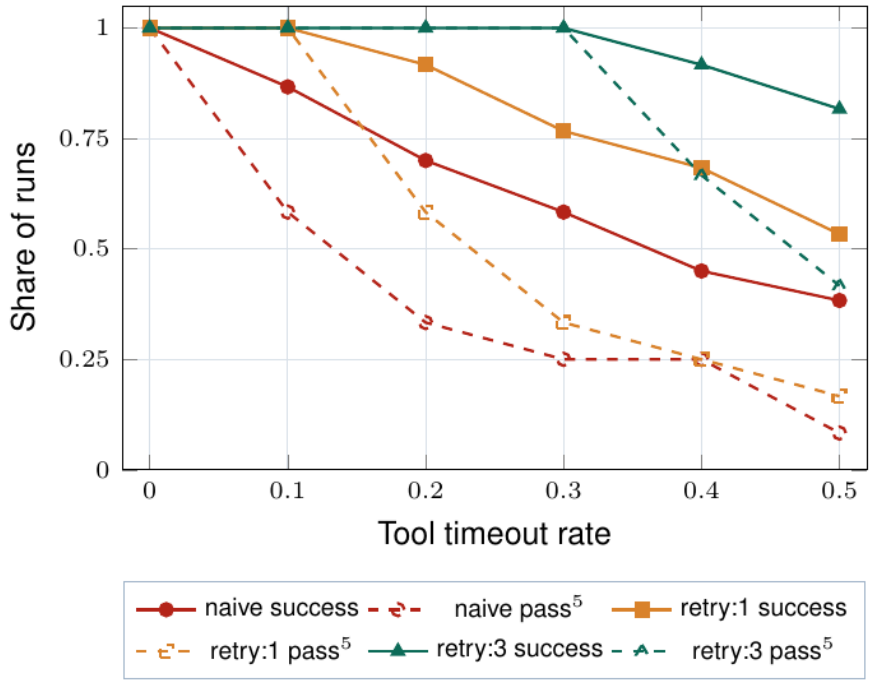
retry:3 vs retry:1 on 60 paired runs: retry:3 alone passed 14, retry:1 alone passed 0; McNemar p = 0.0001 (significant)
```

Terminal

```
$ pytest -q
.............................................................. [ 88%]
.........                                                     [100%]
81 passed in 0.70s
$ ruff check src tests examples
All checks passed!
```

- 30% of tool calls time out, 12 tasks, 5 trials each
- Naive agent: 0.58 success but pass^5 only 0.25
- Three retries: 1.00 success, about 2x cost and latency
- McNemar p = 0.0001: the difference is real

Preliminary Results



Success (solid) and pass^5 (dashed) as timeouts rise

```
Terminal

$ relai selfcheck --trials 5
diagnosed root cause per agent, reliability suite, timeout rate 0.3
agent     planted      planning  tool_misuse  context  recovery  horizon  unclassified  none  agreement
--------  -----------  --------  -----------  -------  --------  -------  ------------  ----  ---------
naive     recovery     0         0            0        25        0        0             35    1.00
retry     (control)    0         0            0        1         11       0             48    0.80
guessing  planning     55        0            0        0         0        0             5     1.00
looping   context      0         0            55       0         0        0             5     1.00
careless  tool_misuse  0         60           0        0         0        0             0     1.00

precision by mode: context 1.00, planning 1.00, recovery 1.00, tool_misuse 1.00
```

- Classifier self-check: every planted defect is found, precision 1.00 for all four modes
- Fault injector matches theory: success (1 - p)^L, e.g. 0.33 measured vs 0.34 expected for 3 calls
- Sweep: 2 retries is the cheapest setting with success >= 0.95

Research Paper and Work Status

First research paper (IEEE format)

Agent Reliability Lab: Controlled Fault Injection and Evidence-Based Failure Analysis for Tool-Using Language Agents

- Done: Abstract, Introduction, Related Work, Problem Statement, Proposed Methodology
- Added: preliminary results with 3 tables and 3 figures
- Next: live LLM results, then submission

Done

- v0.2 package: 17 modules, 81 tests, CI, walkthrough site
- SRS, class, sequence, DFD, ER, Gantt

Next (Oct to Dec)

- Live LLM study: hosted and local models
- Suites adapted from public benchmarks
- More fault kinds: rate limits, corrupted outputs
- Semester VIII: dashboard, trace import, release 1.0

References

[1] S. Yao, J. Zhao, D. Yu et al., "ReAct: Synergizing Reasoning and Acting in Language Models", ICLR, 2023.

[2] S. Yao, N. Shinn, P. Razavi, K. Narasimhan, "tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains", arXiv:2406.12045, 2024.

[3] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, A. Narayanan, "AI Agents That Matter", arXiv:2407.01502, 2024.

[4] S. Kapoor et al., "Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation", ICLR, 2026.

[5] S. Rabanser, S. Kapoor, P. Kirgis et al., "Towards a Science of AI Agent Reliability", ICML, 2026.

[6] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?", arXiv:2503.13657, 2025.

[7] "ToolMisuseBench: An Offline Deterministic Benchmark for Tool Misuse and Recovery in Agentic Systems", arXiv:2604.01508, 2026.

[8] "AgentChaos: Chaos Engineering for Agent Systems via Programmatic Fault Injection", arXiv:2608.06790, 2026.

[9] M. Chen et al., "Evaluating Large Language Models Trained on Code", arXiv:2107.03374, 2021.

[10] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages", Psychometrika, 1947.

[11] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference", J. Amer. Statist. Assoc., 1927.

[12] A. Basiri et al., "Chaos Engineering", IEEE Software, vol. 33, no. 3, 2016.

Thank You

Feedback from the Panel

- The panel wants the team to :